LAPTOP SETUP & OPERATIONS GUIDE

EPtask Ops Console v2.0

Install and run the complete live application on your own machine: the Next.js operations console and the FastAPI service that executes the EPtask V7.1 notebook on every request — with real OpenStreetMap roads, side-by-side warning-network comparison, persistent job history and optional API-key authentication.

Applies to	EPtask-Ops-Console-v2.zip
Stack	Python 3.10 - 3.12 · Node.js 20+ · SQLite
Date	October 2026

Contents

1. What you are installing 1

2. Prerequisites and system requirements 2

3. Installing the simulation service 2

4. Installing and building the console 3

5. A two-minute tour of the console 4

6. Configuration reference 4

7. Offline behaviour and data sources 5

8. Troubleshooting 6

9. Tests and measured performance 7

1. What you are installing

This guide installs the complete EPtask operations console on your own laptop: a professional Next.js frontend together with a FastAPI service that executes the original EPtask V7.1 research notebook code on every request. Nothing in the simulation pages is pre-computed or replayed - the corridor trials, the vehicular edge-computing cell and the paired outage studies all run live, with the notebook's own algorithms, parameters and trained policy exports. Two local processes are all that is required, and no cloud service, account or third-party key is involved at any point.

The delivery is version 2 of the console, which adds four upgrades over the first release. Real OpenStreetMap roads can now replace the notebook's schematic map: the service fetches drivable highways for the corridor from the Overpass API, caches the extract on disk and rebuilds the road graph through the notebook's own **build_corridor()** function, which keeps the real roads only when they are connected and a detour exists, and otherwise falls back to the schematic with the reason displayed. A new Compare panel lets you adjust a hazard preset - footprint, traffic intensity, blockage seed, dispatch time - and run two warning networks side by side, with two maps, informed-share curves and a per-metric delta computed from live executions. Run history is now persistent: every job, including its full result, is stored in a local SQLite database and survives restarts. Finally, an optional API-key authentication layer can lock the HTTP surface when the machine is shared.

```
browser (Chrome / Edge / Firefox)
|
v
frontend Next.js 16 console port 3000
| Overview - Corridor 3D map - Compare networks
| VEC cell - Outage study - Reference - About
| HTTP, direct connection (CORS enabled)
v
backend FastAPI + onnxruntime port 8100
core/ original eptask notebook modules (unmodified)
assets/ graph, ONNX models, reference results, OSM extract
app/ services, jobs + SQLite history, auth, OSM fetch
```

2. Prerequisites and system requirements

The console is a plain desktop application stack: Python for the simulation service and Node.js for the frontend build. There is no database server to install - the v2 job history uses the SQLite library built into Python - and no container runtime. The list below is deliberately conservative; if your machine runs a modern browser comfortably it will run the console.

Requirement	Minimum	Notes
Operating system	Windows 10/11, macOS 12+, Linux	anything that runs Python and Node
Python	3.10 - 3.12 (3.12 recommended)	on Windows, tick Add python.exe to PATH in the installer
Node.js	20 LTS or newer	npm is included; bun 1.1+ also works
Disk space	about 5 GB	the PyTorch CPU wheel is roughly 900 MB unpacked
Memory	8 GB recommended	the OSM road-graph build peaks around 2 - 3 GB
Internet	first install and live OSM fetching only	simulations run fully offline

Table 1. Prerequisites. The app itself runs offline after installation.

3. Installing the simulation service

Unzip the delivery to a folder without spaces in its path, for example **C:\EPtask** on Windows or **~/eptask** on macOS and Linux. Open a terminal in **mini-services/eptask-api** - that folder is the backend. Create a virtual environment so the notebook's dependencies stay isolated from your system Python, then install them. The install downloads PyTorch (CPU build), onnxruntime, FastAPI, networkx and a few smaller packages; on a normal connection this takes five to ten minutes, most of it PyTorch.

```
# macOS / Linux
cd mini-services/eptask-api
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
# start the service
bash run.sh
```

```
# Windows PowerShell
cd mini-services\eptask-api
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

```
# start the service
python -m uvicorn main:app --host 0.0.0.0 --port 8100
```

Terminal 1 - the simulation service on port 8100.

Wait for the line **Application startup complete.** - the service loads the notebook's graph, latency samples, reference results and model exports in one to two seconds. If Windows refuses to activate the virtual environment because script execution is disabled, run **Set-ExecutionPolicy -Scope CurrentUser RemoteSigned** once and retry; this is a PowerShell default, not a problem with the app. If the PyTorch download stalls, install it separately from the CPU wheel index (**pip install torch --index-url https://download.pytorch.org/whl/cpu**) and re-run the requirements install, which will then finish in seconds.

Verify the service before moving on. Opening **http://localhost:8100/health** in a browser should return a JSON object with **"status": "ok"**, the loaded graph size and the cached model list; **http://localhost:8100/docs** shows the interactive API documentation generated from the same code that serves the console. Leave this terminal running - the backend must stay up while you use the console.

4. Installing and building the console

Open a second terminal in the project root - the folder that contains **package.json**. The frontend is a standard Next.js application: install its dependencies, produce the production build, and start it. The shipped **.env.local** already points the browser at **http://localhost:8100**, so no configuration is needed on a laptop.

```
# from the project root
npm install # 2 - 4 minutes
npm run build # production build (recommended)
npm run start # serves the console on http://localhost:3000
# ...or, for development with hot reload:
npm run dev
```

Terminal 2 - the operations console on port 3000.

Open **http://localhost:3000** in any modern browser. Within a few seconds the header should show **simulation service online** with a green indicator; if it instead reports the service as unreachable, the backend from section 3 is not running, or the frontend was built before the **.env.local** file existed - start the backend and rebuild. The first page load after a fresh build is slower while the browser downloads the JavaScript bundles; subsequent navigation is immediate.

To stop the application, press Ctrl+C in each terminal. The SQLite run history, the OSM road cache and the logs all survive; the next start picks up exactly where you left off.

5. A two-minute tour of the console

The console is organised into seven sections, each tied to a specific part of the notebook. The tour below follows the natural workflow: inspect the system, run the road-scale scenario, compare warning networks, then drill into the micro-scale cell and the outage study.

Tab	What it does	What to try first
Overview	provenance, service health, model registry, live-vs-static map	check the health panel and the run identifier
Corridor - N1	3D map; runs corridor.run_trial live for the ambulance and convoy	press Run simulation; then Load real OSM roads and run again on real roads
Compare networks	hazard preset editor + two warning modes side by side	None vs V2V+RSU on Landslide; then tune the footprint slider and rerun
VEC cell	558 m micro-scale cell driven by the shipped ONNX policies	launch model B v30 with the default outage event; open a decision explanation
Outage study	paired C3 study: any policies, outage sizes and seeds, live	run the default A vs B study and compare with the notebook's recorded table
Reference	recorded artifacts from the notebook run, clearly labeled	browse the recorded corridor analysis and figures
About	architecture, fidelity evidence and honest limitations	read the v2 list and the limitations before relying on numbers

Table 2. The seven console sections and a first exercise for each.

Two behaviours are worth knowing about before you explore. First, simulations are queued on a two-worker pool: you can always launch a second study while one runs, and the progress bar names the real work in flight (per convoy member, per tick, per rollout) rather than pretending. Second, on the Corridor tab the **Road network source** card always tells you which road graph is live and why: after loading real OSM roads it shows the graph size, the reason the notebook's checks accepted it, and how far each named place had to be snapped to the nearest mapped road - distances that are themselves an honest quality measure of the data.

6. Configuration reference

Everything is configured through environment variables; there are no config files to edit and no registry entries. Backend variables are read by the simulation service when it starts (the API key is re-read per request); frontend variables are baked in when the frontend is built, so after changing them run **npm run build** again. All of them are optional - the defaults are correct for a laptop.

Variable	Default	Effect
EPTASK_PORT	8100	HTTP port of the simulation service
EPTASK_API_KEY	(unset)	set it to require an API key on every /api call; clients send X-API-Key (or ?api_key= for images); the console shows a key gate
EPTASK_DB	data/jobs.db	SQLite file for the persistent job history
EPTASK_OSM_CACHE	assets/osm_cache	where Overpass extracts are cached
EPTASK_LOG_LEVEL	INFO	DEBUG for verbose logs
EPTASK_CORE / EPTASK_ASSETS	bundled	override the notebook modules / asset locations
NEXT_PUBLIC_API_URL	http://localhost:8100	where the browser finds the API (frontend)
NEXT_PUBLIC_GATEWAY_MODE	(unset)	leave unset on a laptop; sandbox deployments set 1

Table 3. Environment variables. A commented .env.example ships with the zip.

To enable authentication, export a long random string as **EPTASK_API_KEY** before starting the backend, then reload the console: a key gate appears, and the same key pasted once is stored in the browser and sent as the **X-API-Key** header on every request. The **/health** endpoint stays open without a key so that scripts and monitors can still check liveness. Wrong keys are rejected in constant time and logged with a request identifier - never the key itself. To disable authentication again, unset the variable and restart the service.

```
# macOS / Linux - enable auth
export EPTASK_API_KEY="a-long-random-string"
bash run.sh

# Windows PowerShell - enable auth
$env:EPTASK_API_KEY = "a-long-random-string"
python -m uvicorn main:app --host 0.0.0.0 --port 8100
```

7. Offline behaviour and data sources

The console distinguishes three kinds of data, and the interface labels each one everywhere it appears. Live data is computed at request time: corridor trials, convoy behaviour, cell rollouts, outage studies and side-by-side comparisons. Static reference data is the notebook run's own recorded bundle - the results tables and figures in the Reference tab - served as-is and never presented as a live result. Assumptions inherited from the sources (hazard footprints, dispatch times, road speeds, fixed radio ranges) are declared in code and in the About panel.

Real OpenStreetMap roads are live but network-dependent, and the delivery is prepared for both cases. A 17 MB Overpass extract of the corridor ships inside the zip, so the **Load real OSM roads** button works offline on first use; **Re-fetch** downloads fresh map data through four public Overpass mirrors and needs internet access. When every mirror is unreachable, the service keeps the current graph and reports the exact fetch

errors in the source card - it never silently substitutes the schematic. The cached extract and the bundled one keep the OSM mode usable on a permanently offline machine. NASA COOLR landslide points remain the bundled sample because the live dataset requires an Earthdata login that a laptop deployment does not have; the console labels them accordingly.

8. Troubleshooting

Most problems below are environmental rather than defects, and each has a one-line diagnosis. The backend log (the first terminal) and the browser console are the two places to look when the table does not cover a case; both produce structured, actionable messages by design.

Symptom	Cause and fix
Header says simulation service unreachable	backend not running, or frontend built before .env.local existed; start the backend and rebuild
pip install torch is slow or fails	use the CPU wheel: pip install torch --index-url https://download.pytorch.org/whl/cpu, then reinstall requirements
Port 8100 or 3000 already in use	change the port (EPTASK_PORT, and NEXT_PUBLIC_API_URL in .env.local, then rebuild) or free the port
Windows blocks venv activation	Set-ExecutionPolicy -Scope CurrentUser RemoteSigned, then retry
Firewall prompt on first start	allow on private networks; for a strictly local machine start uvicorn with --host 127.0.0.1
3D map is empty or grey	WebGL2 unavailable; tables and charts stay usable - enable hardware acceleration or use Chrome/Edge/Firefox
OSM fetch reports mirror failures	network blocks Overpass; the bundled extract is used automatically, retry Re-fetch later
First OSM load takes 20 - 60 s	normal: about 160 k nodes are built and analysed once; runs on OSM roads then take 3 - 8 s per trial
Auth gate loops	the key in the browser must match EPTASK_API_KEY exactly; clear it (Cancel) and re-enter, or unset the variable
History disappeared	data/jobs.db was deleted; history resets, everything else is unaffected

Table 4. Environmental issues and their one-line fixes.

9. Tests and measured performance

The delivery includes a 55-case pytest suite that you can - and should - run on your machine after installing the backend. It covers asset integrity, the HTTP surface, deterministic reproduction of the notebook's recorded traces (byte-identical ambulance and civilian tracks across all twelve corridor presets, and the full 520-tick cell trace), input validation, the v2 upgrades specifically (authentication, persistence across restart, side-by-side comparison, OSM ingestion and offline fallback), and the end-to-end primary workflow.

```
cd mini-services/eptask-api
python -m pytest tests/ -q # 55 tests, about 3 minutes
```

Expected result: 55 passed.

Operation	Typical wall time
Backend startup (asset load)	1 - 2 s
Corridor run, schematic roads, no convoy	1 - 2 s
Corridor run, schematic roads, with convoy (7 trials)	8 - 14 s
Corridor run on real OSM roads, with convoy	40 - 70 s
Side-by-side comparison (2 live runs, no convoy)	4 - 8 s
VEC cell, 520 ticks	1 - 2 s
Paired outage study (2 policies x 3 sizes x 3 seeds)	6 - 10 s
OSM graph rebuild from the bundled cache	15 - 30 s
OSM graph rebuild with a fresh Overpass fetch	+ 10 - 60 s network time

Table 5. Measured on 4 vCPU; scale roughly with core count for the convoy studies.

Two simulations may run concurrently; further requests queue and the UI shows the real stage of the work. If a laptop has less than 8 GB of RAM, prefer the schematic roads for convoy-sized studies - the OSM graph build is the only step with a multi-gigabyte peak, and its result stays cached in memory afterwards. When a run contradicts your intuition, that is the console doing its job: every number comes from the notebook's own code, on the road network you chose, and the Reference tab is always there to separate a recorded artifact from a live result.